

Llenguatges de Programació, FIB, 18 de juny de 2020

Es valorarà la concisió, claredat i brevetat a més de la completesa i l'exactitud de les respostes.
Tots els problemes puntuen igual.

1. λ -càlcul

1. Considereu les funcions següents:

$$S = \lambda f g x. f x (g x)$$

$$K = \lambda x y. x$$

$$I = \lambda x. x$$

Demostreu que $S K K = I$.

2. Recordeu que, al λ -càlcul, els valors fals i cert es defineixen amb $F = \lambda x y. y$ i $T = \lambda x y. x$ respectivament.

Trobeu una definició ben senzilla per la funció C (abreviatura de condicional) que, aplicada a tres valors, retorni el segon si el primer és cert i retorni el tercer si el primer és fals (és a dir, un *if-then-else*).

Mostreu que la vostra definició és correcta.

2. Haskell

Digueu, de forma raonada i utilitzant reescriptura, quan val *unicorn*:

```
unicorn = do
  x <- [1, 2, 3]
  y <- [4, 5]
  return x
```

Compte! La resposta correcta no és *unicorn* = [1, 2, 3]. Recordeu que les llistes són mònades. Concretament:

```
instance Monad [ ] where
  return x = [x]
  xs >>= f = concat (map f xs)    -- = concatMap f xs
```

3. Inferència de tipus

Inferiu el tipus més general de la funció *rainbow* sabent que *frozen* :: **Int** → **Int**.

```
rainbow x = do
  y <- x
  return (frozen y)
```

Per a fer-ho, dibuixeu el seu arbre de sintàxi, etiqueteu els nodes amb els seus tipus i genereu les restriccions de tipus i classes. Resoleu-les per obtenir la solució i assenyaieu el resultat final amb un requadre.

4. Compilació

Es vol estendre la pràctica d'aquest curs (SkylineBot) per incorporar dos nous operadors:

- $s_1 - s_2$

Aquest operador substreu l'skyline s_2 a l'skyline s_1 , i té la mateixa prioritat que l'operador de suma.

- $s[x_1:x_2]$

Aquest operador selecciona el segment de l'skyline s que està entre les posicions x_1 i x_2 . També es permet aplicar l'operador quan una de les dues posicions sigui absent (però no les dues), amb la semàntica habitual d'indexació de vectors: $s[:x_2]$ selecciona l'skyline s desde l'inici fins a la posició x_2 , i $s[x_1:]$ selecciona l'skyline s desde el final fins a la posició x_1 .

Feu una extensió al llenguatge d'operacions sobre skylines per poder parsejar els dos operadors anteriors. Heu d'incloure tot el vostre fitxer de gramàtica (.g) ressaltant-hi clarament (per exemple, amb colors o comentaris) els canvis efectuats.

5. Subtipos en C++

Como ya debéis saber, un constructor de tipos C es covariante si cuando T_1 es subtipo de T_2 , se tiene que $C\langle T_1 \rangle$ es subtipo de $C\langle T_2 \rangle$. C es contravariante si cuando T_1 es subtipo de T_2 , se tiene que $C\langle T_2 \rangle$ es subtipo de $C\langle T_1 \rangle$. Y C es invariante si no es ni covariante ni contravariante.

1. ¿Podemos considerar covariante el constructor **vector** $\langle T \rangle$?

En caso que creáis que sí, argumentad por qué lo creéis. Si creéis que no, poned un ejemplo de una función f con (al menos) un parámetro de tipo **vector** $\langle T_2 \rangle$ que nos pudiera dar problemas si **vector** fuera covariante.

2. ¿Podemos considerar contravariante el constructor **vector** $\langle T \rangle$?

En caso que creáis que sí, argumentad por qué lo creéis. Si creéis que no, poned un ejemplo de una función f con (al menos) un parámetro de tipo **vector** $\langle T_1 \rangle$ que nos pudiera dar problemas si **vector** fuera contravariante.

6. Python

Si executem el següent programet, el resultat és [1, 2, 3].

```
def func1():  
    def f1(i):  
        t = i  
        return lambda: t  
  
    t = 0  
    return [f1(i) for i in [1, 2, 3]]  
  
fs = func1()  
print([f() for f in fs])
```

En canvi, si executem aquest altre programet, el resultat és [3, 3, 3].

```
def func2():  
    def f2(i):  
        nonlocal t  
        t = i  
        return lambda: t  
  
    t = 0  
    return [f2(i) for i in [1, 2, 3]]  
  
fs = func2()  
print([f() for f in fs])
```

Expliqueu per què es dona aquesta diferència.